KrishiMitra AI

Complete Project Guide & Handover Manual

A step-by-step technical, installation, operation, testing and maintenance guide

Project status: Development checkpoint prepared for handover

Documentation date: 28 September 2026

Technology: React + Vite + TypeScript, FastAPI + Python, PostgreSQL, SQLAlchemy, Ollama, ML services

Purpose of this document

This manual is designed so a new developer can install KrishiMitra AI on a fresh Windows machine, understand how the system works, run every major module, troubleshoot common problems, and safely modify the project without needing the original development conversation.

Important: This guide intentionally does not contain real passwords, API keys, JWT secrets, or the local backend .env file. Those values must remain private.

1. Project Overview

KrishiMitra AI is an agricultural intelligence platform intended to help farmers manage farm information and receive weather-aware, crop-aware and AI-assisted guidance. The current application combines a React frontend, a FastAPI backend, PostgreSQL persistence, machine-learning services, live weather integration and a local Ollama language model.

The implemented product areas include:

- Dashboard and farm overview
- My Farm profile and farm data management
- Crop management and AI crop recommendation
- Crop disease image detection
- Fertilizer recommendation
- Irrigation advisory and irrigation records
- Weather and five-day forecast
- Market price information
- Government agricultural schemes
- Crop calendar and activity planning
- AI Assistant using local Ollama plus farm/weather context
- Notifications
- User profile and settings
- English, Hindi and Marathi UI support

2. Current Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript	Web application UI
Build	Vite 8	Development server and production build
Routing	React Router	Page navigation
Data fetching	TanStack React Query	API state and caching
UI	Tailwind CSS + Lucide	Styling and icons
i18n	i18next + react-i18next	English/Hindi/Marathi UI
Backend	FastAPI + Python	REST API and application logic
ORM	SQLAlchemy	Database access
Database	PostgreSQL	Persistent application data
Authentication	JWT + Argon2/password hashing	User authentication
AI runtime	Ollama	Local LLM inference
LLM	llama3.1:8b	Current configured assistant model
ML	scikit-learn / PyTorch ecosystem	Crop/disease/fertilizer intelligence
Weather	Visual Crossing API	Farm weather and forecast

3. Complete Architecture

High-level request flow:

```
Browser
|
v
React + Vite Frontend
|
| HTTP / JSON + Bearer token
v
FastAPI Backend
|---- Authentication / JWT
|---- Farm & Crop services
|---- Weather service
|---- Market / Schemes / Notifications
|---- ML services
|---- AI Assistant
|
+---- PostgreSQL
|
+---- Visual Crossing Weather API
|
+---- Ollama (localhost:11434)
|
+---- llama3.1:8b
```

The frontend should never contain database credentials or private API keys. The backend owns database access, authentication, weather credentials and AI integration.

4. Project Folder Structure

```
KrishiMitra_AI/
├── backend/
│   ├── app/
│   │   ├── ai/
│   │   ├── api/
│   │   │   └── routes/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   └── main.py
│   ├── requirements.txt
│   ├── .env
│   └── venv/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── lib/
│   │   ├── pages/
│   │   ├── routes/
│   │   └── i18n.ts
│   ├── package.json
│   └── vite.config.ts
├── database/
│   ├── README.md
│   ├── schemas/
│   └── seeds/
├── ml/
├── scripts/
├── docs/
├── .env.example
├── .gitignore
```

```
├── docker-compose.yml
└── README.md
```

5. Frontend Architecture

The frontend is a Vite-powered React application. Pages are separated by feature and use hooks/services to communicate with the FastAPI backend. Translation resources are centralized in frontend/src/i18n.ts.

Location	Responsibility
frontend/src/pages/	Feature pages such as Dashboard, Weather, Irrigation, Fertilizer, Market and AI Assistant
frontend/src/components/	Reusable UI components
frontend/src/hooks/	Reusable API/data-fetching hooks
frontend/src/routes/	Application route definitions
frontend/src/i18n.ts	English, Hindi and Marathi translation resources
frontend/package.json	Node dependencies and npm scripts
frontend/vite.config.ts	Vite build configuration

6. Main Frontend Pages

Page	Main purpose
Dashboard	Farm summary, active crops, weather overview and quick actions
My Farm	Farm profile, location, soil, irrigation and water-source information
Weather	Current conditions and five-day farm forecast
Crops	Crop records plus AI crop recommendation
Disease Detection	Upload a crop image and receive disease prediction
Fertilizer	Generate fertilizer guidance from crop/soil nutrient values
Irrigation	Irrigation recommendations and irrigation records
Market	Agricultural market price information
Schemes	Government agricultural schemes with filters and official portals
Crop Calendar	Sowing, irrigation, fertilizer, monitoring and harvest activities
AI Assistant	Chat with KrishiMitra AI using farm and weather context
Notifications	View and manage farmer notifications
Profile	View account information
Settings	Language and application preference controls

7. Backend Architecture

FastAPI is the central application layer. API routes validate requests, enforce authentication where required, call services/ML/AI components, and return structured JSON responses.

Route group	Purpose
/auth	Registration and login
/users	Current-user profile
/farms	Farm CRUD and ownership checks
/farms/{farm_id}/crops	Crop CRUD
/crop-recommendation	AI/ML crop recommendation
/disease-detection	Image-based disease prediction
/fertilizer	Fertilizer recommendation
/farms/{farm_id}/irrigations	Irrigation CRUD
/farms/{farm_id}/irrigation/recommend/{crop_id}	Irrigation recommendation
/market	Market price lookup
/notifications	Notification listing and read state
/schemes	Government scheme listing and management
/farms/{farm_id}/weather	Weather and forecast
/ai/chat	AI Assistant chat

8. Important API Operations

Method	Endpoint	Authentication
POST	/auth/register	No
POST	/auth/login	No
GET	/users/me	Yes
GET/POST/PUT/DELETE	/farms	Yes
GET/POST/PUT/DELETE	/farms/{farm_id}/crops	Yes
POST	/crop-recommendation/predict	Yes
POST	/disease-detection/predict	Yes
POST	/fertilizer/recommend	Yes
GET	/market/prices	Yes
GET	/schemes	Yes
GET	/notifications	Yes
POST	/notifications/{id}/read	Yes
POST	/notifications/read-all	Yes
POST	/ai/chat	Yes
GET	/farms/{farm_id}/weather	Yes

9. Database Architecture

SQLAlchemy models currently define the main persistent entities. The database design is centered on users → farms → crops, with supporting irrigation, market, scheme and notification data.

Model/table	Important data
users	Name, email, password hash, active status, timestamps
farms	User, farm name, area, village/taluka/district/state, pincode, latitude/longitude, soil, irrigation and water sourc
crops	Farm, crop name, variety, area, sowing date, harvest date, season, status
irrigations	Farm/crop, date, water quantity, duration, method, status and source
market_prices	Commodity, market, district/state, min/max/modal price, unit and date
schemes	Scheme information, category, level, state, beneficiaries, eligibility, benefits, documents and official URL
notifications	User, optional scheme, type, title/message, read state and creation time

10. AI Assistant Architecture

The AI Assistant is local-first. The backend receives the farmer's question, builds farm context from the database, attempts to obtain weather context, combines that information with an agricultural system prompt, and sends the final prompt to Ollama.

```
Farmer question
|
v
POST /ai/chat
|
v
ask_assistant(...)
|
+--> build_farm_context(...)
|
+--> find user's latest farm
| |
| +--> build_weather_context(farm)
|
v
Combined farm + weather + optional user context
|
v
SYSTEM_PROMPT + farmer question
|
v
Ollama /api/generate
|
v
llama3.1:8b
|
v
AI response -> frontend
```

The current Ollama integration is configured in backend/app/ai/ollama.py. The assistant orchestration is in backend/app/ai/assistant.py. Farm context is built in backend/app/ai/rag.py and weather context in backend/app/ai/weather_context.py.

Current model configuration:

```
OLLAMA_BASE_URL = "http://localhost:11434"
OLLAMA_MODEL = "llama3.1:8b"
POST /api/generate
stream = false
timeout = 120 seconds
```

The assistant prompt instructs the model to provide practical Indian agricultural guidance, use farm/weather context when available, avoid inventing weather/farm information, state missing information, and recommend expert help for serious disease or chemical-use decisions.

11. Crop Recommendation

The crop recommendation API accepts soil and weather inputs including N, P, K, temperature, humidity, soil pH and rainfall. The backend calls the configured ML model/service and returns the recommended crop.

12. Disease Detection

The disease detection endpoint accepts a multipart image upload, validates the image and passes it to the disease model. The frontend presents the prediction result to the farmer.

13. Fertilizer Recommendation

The fertilizer endpoint uses crop and soil nutrient values including N, P, K and pH to produce a fertilizer recommendation through the backend recommendation engine.

14. Irrigation

Irrigation has both recommendation and record-management functionality. Farm and crop information can be combined with weather context to support irrigation decisions. Actual field decisions should still account for soil moisture and local conditions.

15. Fresh Windows Installation

Use the following process on a new Windows machine. The exact package versions should be taken from the committed project files rather than installing arbitrary latest versions.

Step 1 — Install prerequisites

- Git
- Node.js / npm
- Python
- PostgreSQL
- Ollama
- A modern browser such as Chrome or Edge

Step 2 — Obtain the project

```
git clone <repository-url>
cd KrishiMitra_AI
```

If the project is delivered as a ZIP, extract it to a simple path such as C:\KrishiMitra_AI.

Step 3 — Create backend virtual environment

```
cd C:\KrishiMitra_AI\backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If PowerShell blocks activation, use an appropriate execution-policy adjustment for the current user or run the environment's Python directly.

Step 4 — Configure backend environment

```
DATABASE_URL=postgresql+psycopg://<user>:<password>@localhost:5432/krishimitra_ai
JWT_SECRET_KEY=<long-random-secret>
WEATHER_API_KEY=<Visual-Crossing-key>
WEATHER_API_BASE_URL=https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline
```

Never copy the real .env file into a public repository or handoff ZIP. Use backend/.env.example as the template.

Step 5 — Prepare PostgreSQL

1. Install and start PostgreSQL.
2. Create a database named krishimitra_ai.
3. Create or use the PostgreSQL user specified in DATABASE_URL.
4. Apply the project's database initialization/migration process.
5. Load seed data if the project distribution includes seed SQL intended for the current version.

The SQLAlchemy models are the source of truth for the current application entities. Do not assume an empty SQL file is a complete schema.

Step 6 — Install Ollama and model

```
ollama list
ollama pull llama3.1:8b
ollama run llama3.1:8b
```

Ollama should be reachable at http://localhost:11434. The current backend is configured for llama3.1:8b.

16. Start the Application

Backend terminal:

```
cd C:\KrishiMitra_AI\backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --reload
```

Expected backend address: http://127.0.0.1:8000

Frontend terminal:

```
cd C:\KrishiMitra_AI\frontend
npm install
npm run dev
```

Expected frontend address: http://localhost:5173

17. Production Build Check

```
cd C:\KrishiMitra_AI\frontend
npx tsc --noEmit
npm run build
```

The latest verified build completed successfully. The build reported optimization warnings about a large JavaScript chunk and a future Vite config-loader issue; these were warnings rather than build failures.

18. End-to-End Test Checklist

Area	Test
Authentication	Register a test user, log in and confirm protected pages work
Farm	Create/edit farm details and confirm ownership
Crops	Add, edit and delete a crop
Crop AI	Submit valid soil/weather values and confirm a recommendation
Disease	Upload a valid crop image and confirm prediction
Fertilizer	Submit crop and N/P/K/pH values
Irrigation	Request recommendation and create/read irrigation records
Weather	Configure farm coordinates and confirm current/forecast data
Market	Search market information
Schemes	Search/filter schemes and open official portal links
Calendar	Confirm active crops with dates generate activities
AI	Ask a farm question and confirm Ollama response
Notifications	Open notifications, mark one/all as read
Profile	Verify account information
Settings	Change language and verify persistence
Languages	Test English, Hindi and Marathi across major pages

19. Multilingual Support

The UI uses i18next and react-i18next. The currently implemented languages are English, Hindi and Marathi. The selected language is stored in browser localStorage under krishimitra_language.

Primary file:

```
frontend/src/i18n.ts
```

When adding visible text to a page:

1. Add a translation key to the English resource.
2. Add the same key to the Hindi resource.
3. Add the same key to the Marathi resource.
4. Use t("yourKey") in the component instead of hardcoded UI text.
5. Run TypeScript and production build checks.

20. Where to Make Changes

Requirement	Start here
Change page UI	frontend/src/pages/
Reusable UI	frontend/src/components/
API fetching	frontend/src/hooks/ and frontend API helpers
Routes/navigation	frontend/src/routes/ and layout
Translations	frontend/src/i18n.ts
AI behavior/prompt	backend/app/ai/assistant.py
Ollama model/connection	backend/app/ai/ollama.py
Farm RAG context	backend/app/ai/rag.py
Weather AI context	backend/app/ai/weather_context.py
API endpoints	backend/app/api/routes/
Business logic	backend/app/services/
Database models	backend/app/models/
Request/response schemas	backend/app/schemas/
Authentication	backend/app/api/routes/auth.py and auth services
Environment settings	backend/app/core/config.py and backend/.env
ML implementation	backend ML modules / ml/ project area

21. How to Add a New Page

1. Create the page component in frontend/src/pages/.
2. Add all visible text as translation keys.
3. Create or reuse API hooks.
4. Add the route in the application's route configuration.

5. Add navigation/sidebar entry if required.
6. Test authentication and loading/error states.
7. Test English, Hindi and Marathi.
8. Run npx tsc --noEmit and npm run build.
9. Commit the change with a focused Git message.

22. Troubleshooting

Frontend does not start

- Confirm Node.js is installed.
- Run npm install in frontend.
- Check that port 5173 is not already occupied.
- Check the terminal for TypeScript/Vite errors.

Backend does not start

- Activate backend/venv.
- Confirm requirements are installed.
- Confirm PostgreSQL is running.
- Confirm backend/.env contains DATABASE_URL, JWT_SECRET_KEY and WEATHER_API_KEY.
- Run the backend from the backend directory.

AI Assistant says Ollama is unavailable

- Run ollama list.
- Confirm llama3.1:8b exists.
- Confirm Ollama is running.
- Test http://localhost:11434/api/tags.
- Check backend/app/ai/ollama.py for the configured model and URL.
- Restart the backend after configuration changes.

Weather is unavailable

- Confirm the farm has latitude and longitude.
- Confirm WEATHER_API_KEY is valid.
- Check the Visual Crossing API response.
- Review the backend terminal for the exact error.

Database connection fails

- Check PostgreSQL service status.
- Confirm database name is krishimitra_ai.
- Verify DATABASE_URL syntax.
- Confirm the PostgreSQL user has access to the database.
- Do not hardcode credentials into source code.

Translations are missing

- Check whether the key exists in all three language resources.
- Confirm the component calls t("key").
- Restart Vite if the development server has stale module state.

- Run the production build to catch duplicate keys or syntax errors.

23. Security Rules

- Never commit backend/.env.
- Never place database passwords in Python source.
- Never place weather API keys in the frontend.
- Use a strong random JWT secret in each real environment.
- Use separate credentials for development and production.
- Do not expose PostgreSQL directly to the public internet.
- Do not publish private farmer information or authentication tokens.

24. Git Development History

The latest project history shows the following major milestones:

Commit	Purpose
2bf4137	Complete KrishiMitra AI core modules
391defa	Add multilingual UI support
1df791a	Translate Weather page
5d6418c	Translate Irrigation page
1d70c53	Translate Fertilizer page
f2c1b90	Translate Market page
d7011ec	Translate Crop Calendar page
7913d09	Translate Crop Recommendation page
02ec3b0	Translate Disease Detection page
5d95b54	Translate AI Assistant page
aa1b6b8	Translate Notifications page
058ef2f	Add notification translations
de8ac6b	Translate Profile page
3125908	Translate Schemes page
57f3003	Translate Settings page
7126af3	Resolve multilingual build errors
c1a6c72	Correct AI Assistant dashboard route
1d85ec0	Move database credentials to environment

25. Recommended Change Workflow

1. Pull or copy the latest project version.
2. Create a focused change.
3. Run the relevant local test.
4. Run npx tsc --noEmit.
5. Run npm run build for frontend changes.
6. Test the affected backend endpoint for backend changes.
7. Review git diff.
8. Commit with a clear message.
9. Do not commit secrets, venv, node_modules or generated caches.

26. Current Known Limitations / Practical Notes

- The AI Assistant depends on a locally running Ollama service and the configured model.

- AI responses are generated language-model output and should not be treated as a substitute for field-level agricultural expertise.
- Weather quality depends on valid farm coordinates and the external weather service.
- The frontend production build currently emits a large-chunk optimization warning.
- Some UI fallback strings remain hardcoded in places and should be converted to translation keys during future polishing.
- The project should be re-tested on the target sister's machine because Windows, PostgreSQL, Python, Node and Ollama installations can differ.

27. Handover Checklist

- Clean Git working tree
- Real .env excluded from ZIP
- Safe .env.example included
- README included
- This PDF guide included
- Source code included
- Database seed/reference files included
- node_modules excluded
- backend/venv excluded
- __pycache__ and test caches excluded
- Frontend build verified
- Backend configuration verified
- Ollama model documented
- Login/register tested
- Major feature flows tested
- English/Hindi/Marathi tested

28. Quick Start — One-Page Reference

For the person receiving the project, this is the shortest reliable startup sequence after prerequisites and PostgreSQL are configured.

Terminal 1 — Backend

```
cd C:\KrishiMitra_AI\backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --reload
```

Terminal 2 — Frontend

```
cd C:\KrishiMitra_AI\frontend
npm install
npm run dev
```

Ollama

```
ollama list
# Make sure llama3.1:8b is installed and Ollama is running
```

Open the frontend at http://localhost:5173. The backend should be available at http://127.0.0.1:8000.

29. If You Need to Modify This Project, Start Here

For a UI change, start in frontend/src/pages. For a translation change, start in frontend/src/i18n.ts. For an API change, start in backend/app/api/routes. For database structure, start in backend/app/models and the related service/schema. For AI behavior, start in backend/app/ai/assistant.py. For Ollama configuration, start in backend/app/ai/ollama.py. For farm context, start in backend/app/ai/rag.py. For weather context, start in backend/app/ai/weather_context.py.

Before changing a working feature, read the related page, hook, route, service and model together. Make one focused change, test it, then commit it. This reduces the chance of breaking another module.

End of KrishiMitra AI Project Guide